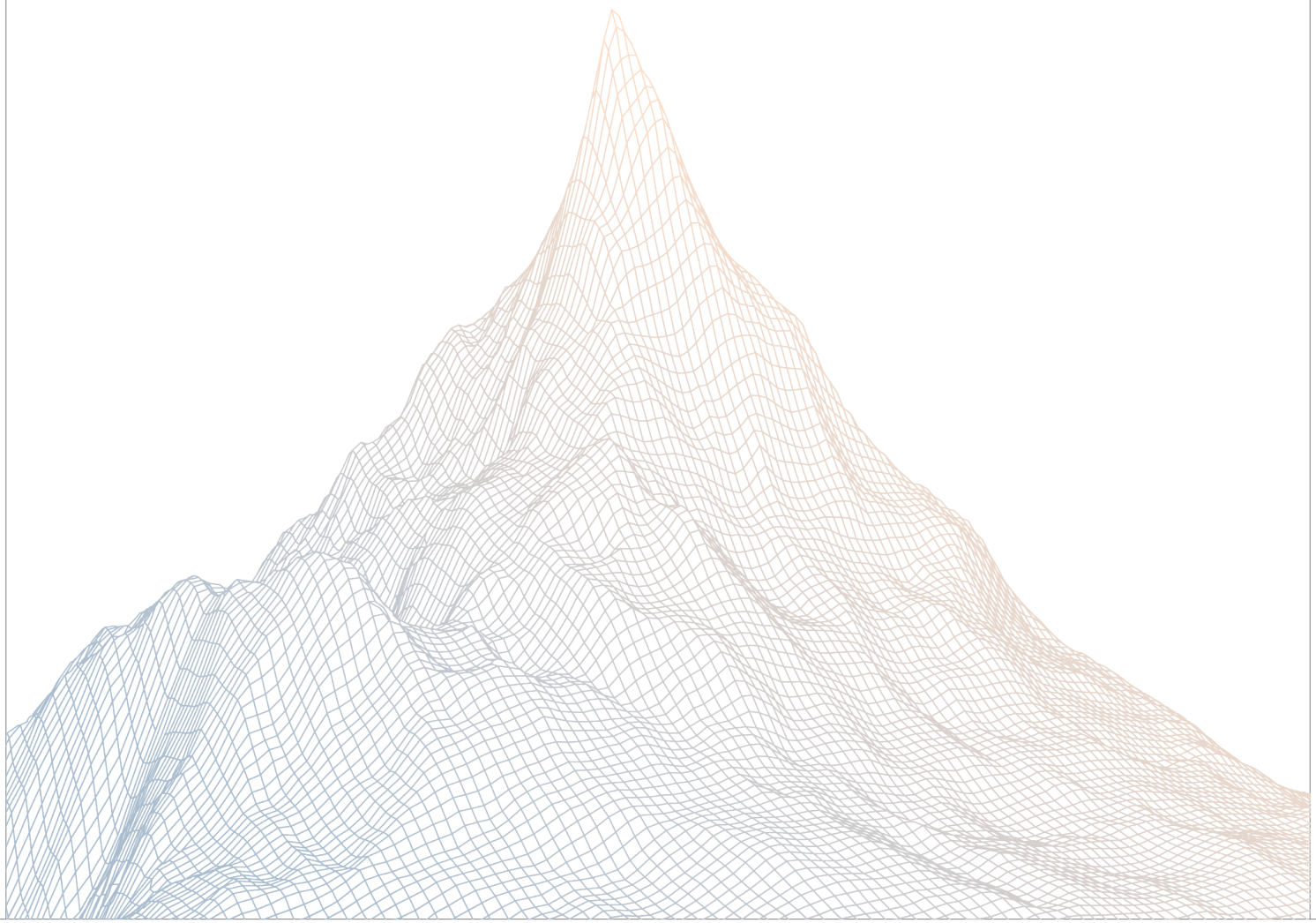
Zenith

Kite AI

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: March 24th to April 3rd, 2026

AUDITED BY: ast3ros
said

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Kite AI 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 7

4.1 High Risk 8

4.2 Medium Risk 12

4.3 Low Risk 17

4.4 Informational 43

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Kite AI

Kite enables AI agents to autonomously transact at scale with cryptographic safety and native x402 compatibility—solving the infrastructure crisis imprisoning the agent economy today.

2.2 Scope

The engagement involved a review of the following targets:

Target	lst-kite
Repository	https://github.com/suzaku-network/lst-kite
Commit Hash	`68170b0c1287f63dd120b0e9487a8795264a6fb9`
Files	`src/contracts/*`

Target	Mitigation Review
Repository	https://github.com/suzaku-network/lst-kite
Commit Hash	`363b322234600c47f3e3d7825735921d4add0338`
Files	`src/contracts/*`

2.3 Audit Timeline

March 24, 2026	Audit start
April 3, 2026	Audit end
April 15, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	1
Medium Risk	2
Low Risk	12
Informational	5
Total Issues	**20**

3

Findings Summary

ID	Description	Status
H-1	Delegation fees accrued after validator removal are lost	Resolved
M-1	Externally initiated validator removal is not adopted in _processOperatorValidatorsForRemoval	Resolved
M-2	Fresh deposits can capture previously earned rewards	Acknowledged
L-1	Admin can retroactively change operator fee split on already-accumulated rewards	Acknowledged
L-2	_processOperatorDelegationsForRemoval incorrectly skips PendingAdded delegations	Acknowledged
L-3	_processOperatorValidatorsForRemoval consumes scan budget for already pending validators	Resolved
L-4	Validator removal initiated before all delegations are removed	Resolved
L-5	Wrong maturity cutoff for validators	Acknowledged
L-6	initiateValidatorRemoval hardcodes includeUptimeProof to false	Acknowledged
L-7	Inconsistent debt reference points in _selectAndRemoveStake can increase exit debt despite over-contribution	Acknowledged
L-8	Operators skipped by caps unfairly accrue exit debt	Resolved
L-9	requestWithdrawal fee deducts vaultAccountedBalance	Resolved
L-10	MAX_OPERATOR_FEE_BIPS cap can block validator registration	Acknowledged
L-11	requestWithdrawal lacks slippage protection	Acknowledged
L-12	Withdrawal amount may exclude unharvested rewards	Acknowledged
I-1	DEFAULT_ADMIN_ROLE can call initiateDelegatorRemoval	Resolved
I-2	Externally removed delegations may be missed in contribution accounting	Acknowledged
I-3	Malicious protocolFeeRecipient can break vault accounting	Acknowledged

ID	Description	Status
I-4	completeValidatorRegistration does not verify validator belongs to the vault	Resolved
I-5	Insolvent vault has no ETH injection mechanism causing permanent deposit freeze with no admin recovery path	Acknowledged

4

Findings

4.1 High Risk

A total of 1 high risk findings were identified.

[H-1] Delegation fees accrued after validator removal are lost

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target

- StakingVaultOperations.sol#L1151
- StakingVaultOperations.sol#L805

Description:

When harvesting rewards, `_harvestOperatorValidators` iterates over the `operatorValidators` enumerable set and calls `claimValidatorRewards` for each active validator:

```
    function _harvestOperatorValidators(
        StakingVaultStorageLib.StakingVaultStorage storage $,
        uint256 operatorIndex,
        uint256 start,
        uint256 batchSize
    ) internal returns (uint256 totalRewards) {
        ...

        for (uint256 j = start; j < end;) {
>>>         bytes32 validationID = $.operatorValidators[operatorAddr].at(j);
            uint256 reward = _callU256(
                mgr,
                abi.encodeWithSelector(
                    StakingVaultStorageLib.SEL_CLAIM_VALIDATOR_REWARDS,
    validationID, false, uint32(0)
                )
            );
            ...
    }
```

However, when a validator's removal is finalized, `_syncValidatorState` removes it from the `operatorValidators` set:

```
    function _syncValidatorState(
        StakingVaultStorageLib.StakingVaultStorage storage $,
        bytes32 validationID,
        uint256 stakeAmount
    ) internal {
        ...
        if (operatorAddr ≠ address(0)) {
            ...
>>>         $.operatorValidators[operatorAddr].remove(validationID);
        }
    }
```

Once a validator is removed from the set, `_harvestOperatorValidators` will never call `claimValidatorRewards` for it again. This becomes a problem because delegators attached to a removed validator may have their removals completed after the validator itself has been finalized. When a delegator removal completes, `_withdrawDelegationRewards` accrues the validator's commission share (the `delegationFeeBips` cut) into `$._redeemableValidatorRewards[validationID]` on the `StakingManager`:

```
    function _withdrawDelegationRewards(
        address rewardRecipient,
        bytes32 delegationID,
        bytes32 validationID
    ) internal returns (uint256, uint256) {
        StakingManagerStorage storage $ = _getStakingManagerStorage();

        uint256 delegationRewards;
        uint256 validatorFees;

        uint256 rewards = $._redeemableDelegatorRewards[delegationID];

        if (rewards > 0) {
            ...

                // Allocate the delegation fees to the validator.
                if (validatorFees > 0) {
>>>                 $._redeemableValidatorRewards[
                        validationID
                    ] += validatorFees;
                    emit DelegationFeesAccrued(
                        validationID,
                        delegationID,
                        validatorFees
```

```
                    );
                }

            ...
    }
```

Because the validator has already been removed from `operatorValidators`, the vault has no mechanism to harvest the fees that have just been deposited into `$._redeemableValidatorRewards`. As a result, all delegation commission fees that accrue after a validator's removal are stranded in the `StakingManager` and irrecoverable by the vault.

Another scenario where stranded reward can occur, When `_withdrawValidationRewards` fails during `completeValidatorRemoval` due to an underfunded `RewardVault`, the entire `_redeemableValidatorRewards` balance, including the validator's own staking rewards calculated at initiation time, not just delegation commission, is silently preserved in SM while `_syncValidatorState` removes the validator from `operatorValidators` in the same tx.

```
    function _withdrawValidationRewards(
        address rewardRecipient,
        bytes32 validationID
    ) internal {
        StakingManagerStorage storage $ = _getStakingManagerStorage();

        uint256 rewards = $._redeemableValidatorRewards[validationID];
        if (rewards == 0) {
            return;
        }

        bool success = _reward(rewardRecipient, rewards);

        // Only clear rewards if distribution succeeded
        // If failed, rewards remain claimable via claimValidatorRewards
>>>     if (success) {
            delete $._redeemableValidatorRewards[validationID];
            emit ValidatorRewardClaimed(validationID, rewardRecipient,
    rewards);
        }
    }
```

The stranded amount is not bounded to a small commission window, cannot be avoided by keeper ordering (the `RewardVault` balance is external and unpredictable), and has no recovery path, the vault lacks a function to call `claimValidatorRewards` for removed validators.

Recommendations:

Introduce a separate tracking mechanism for validators that have been removed from `operatorValidators` but may still have pending delegator removals in flight.

Kite AI: Resolved with `@764d6bd ...` and `@9aa6584 ...`

Zenith: Verified.

4.2 Medium Risk

A total of 2 medium risk findings were identified.

[M-1] Externally initiated validator removal is not adopted in _processOperatorValidatorsForRemoval

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- StakingVaultOperations.sol#L172-L200

Description:

If a validator's removal is initiated externally, directly on KiteStakingManager (admin calls `KiteStakingManager.forceRemoveValidator`), the validator transitions to `PendingRemoved` status on the SM while the vault's `$.validatorPendingRemoval[validationID]` flag remains `false`. The vault has no awareness of this state.

When `_processOperatorValidatorsForRemoval` subsequently scans this validator during a `prepareWithdrawals` call, the `StakingVaultStorageLib.SEL_FORCE_INITIATE_VALIDATOR_REMOVAL` call will revert. The validator's principal is never counted toward `contribution`, causing `prepareWithdrawals` to underestimate the amount of stake being freed.

The vault may initiate more removals than necessary or fail to satisfy the withdrawal target when liquidity is actually already en route.

```
function _processOperatorValidatorsForRemoval(
    StakingVaultStorageLib.StakingVaultStorage storage $,
    address operatorAddr,
    uint256 remainingTarget,
    uint64 maturityCutoff,
    uint256 removalsThisCall,
    uint256 maxScans
) internal returns (uint256 contribution, uint256 removals,
uint256 scans) {
        ...
```

```
            $.validatorPendingRemoval[validationID] = true;
            if (_callBool(
                    mgr,
                    abi.encodeWithSelector(
>>>                     StakingVaultStor-
    ageLib.SEL_FORCE_INITIATE_VALIDATOR_REMOVAL, validationID, false,
    uint32(0)
                    )
                )) {
                uint256 stakeAmount = $.validatorPrincipal[validationID];
                _recordRemovalInFlight($, operatorAddr, validationID,
    stakeAmount, false);
                contribution += stakeAmount;
                unchecked {
                    ++removals;
                }
                emit
    IStakingVaultOperations.StakingVault__ValidatorRemovalInitiated(operatorAddr,
    validationID);
            } else {
                $.validatorPendingRemoval[validationID] = false;
                emit
    IStakingVaultOperations.StakingVault__ValidatorRemovalFailed(validationID,
    operatorAddr);
            }
    }
```

This is in direct contrast to `_processOperatorDelegationsForRemoval`, which correctly handles the case by detecting `status == 3` (PendingRemoved) and adopting the externally-initiated removal:

```
if (status == 3) {
    // Adopt externally-initiated PendingRemoved we haven't tracked
    _recordRemovalInFlight($, operatorAddr, delegationID, principal, true);
    contribution += principal;
    unchecked {
        ++removals;
    }
    emit IStakingVaultOperations.StakingVault__DelegatorRemovalAdopted(
        operatorAddr, delegationID, principal
    );
```

Recommendations:

Mirror the adoption logic from `_processOperatorDelegationsForRemoval` in `_processOperatorValidatorsForRemoval`.

Kite AI: Resolved with `@cf2a475ff4 ...`

Zenith: Verified.

[M-2] Fresh deposits can capture previously earned rewards

SEVERITY: Medium	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: High

Target

- StakingVault.sol#L346-L348

Description:

The `deposit` function calculates a depositor's share allocation using `getTotalPooledStake` as the pre-deposit baseline. However, `getTotalPooledStake` only reflects rewards that have already been harvested and accounted for in `vaultAccountedBalance`. It does not include pending rewards that have earned on validators and delegators but have not yet been claimed via `harvest`. As a result, if a big amount of unharvested rewards exists in the system, a new depositor receives shares priced against an understated pool value, directly benefiting the new depositor.

```
    function deposit(
        uint256 minShares
    ) external payable nonReentrant whenNotPaused returns (uint256 shares) {
        if (msg.value == 0) revert StakingVault__InvalidAmount();

>>>     uint256 preDepositStake = getTotalPooledStake();
        if (totalSupply() > 0 && preDepositStake == 0)
    revert StakingVault__Insolvent();
        shares = _stakeToShares(msg.value, preDepositStake);
        ...
    }
```

This allows a user to front-run a harvest: observe a large pending reward, deposit immediately before it is claimed, call `harvest`, and then withdraw later at the inflated rate to extract yield they did not earn.

Recommendations:

Call `harvest` at the beginning of `deposit` before computing `preDepositStake`.

Kite AI: Acknowledged. We consider keeper and reward frequency management sufficient

to keep the claimable backlog small enough that extractable value is negligible.

4.3 Low Risk

A total of 12 low risk findings were identified.

[L-1] Admin can retroactively change operator fee split on already-accumulated rewards

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVault.sol#L614-L634

Description:

`setOperatorFeeBips()` update `operatorFeeBips` with no preceding `harvest` call. All `harvest`/reward-split functions read `$.operatorFeeBips` at execution time, applying the current rate to ALL accumulated rewards, including those earned under a different fee.

```
function setOperatorFeeBips(
    uint256 bips
) external onlyRole(StakingVaultStorageLib.VAULT_ADMIN_ROLE) {
    if (bips > StakingVaultStorageLib.MAX_OPERATOR_FEE_BIPS) {
        revert StakingVault__InvalidFee(bips);
    }
    uint16 minFeeBips = _getMinimumDelegationFeeBips();
    if (bips < uint256(minFeeBips)) {
        revert StakingVault__InvalidFee(bips);
    }

    StakingVaultStorageLib.StakingVaultStorage storage $
    = StakingVaultStorageLib._getStorage();
    if (bips + $.protocolFeeBips > StakingVaultStorageLib.BIPS_DENOMINATOR)
    {
        revert StakingVault__InvalidFee(bips);
    }

    uint256 oldFee = $.operatorFeeBips;
```

```
    $.operatorFeeBips = bips;

    emit IStakingVault.StakingVault__OperatorFeeUpdated(oldFee, bips);
}
```

Recommendations:

Consider to trigger `harvest` before `setOperatorFeeBips`.

Kite AI: Acknowledged. Enforcing this on-chain would couple an expensive batch operation to a simple setter. We will document the harvest() before setOperatorFeeBips() ordering as mandatory.

[L-2] _processOperatorDelegationsForRemoval incorrectly skips PendingAdded delegations

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- StakingVaultOperations.sol#L1073

Description:

`_processOperatorDelegationsForRemoval` unconditionally skips delegations in the `PendingAdded` state.

```
        function _processOperatorDelegationsForRemoval(
        StakingVaultStorageLib.StakingVaultStorage storage $,
        address operatorAddr,
        uint256 remainingTarget,
        uint64 maturityCutoff,
        uint256 removalsThisCall,
        uint256 maxScans
    ) internal returns (uint256 contribution, uint256 removals,
    uint256 scans) {
        uint256 delLen = $.operatorDelegations[operatorAddr].length();
        for (uint256 j = delLen; j > 0 && contribution < remainingTarget;) {
            ...
            (bool smOk, uint8 status, uint256 smAmount, uint64 startTime) =
                StakingVaultInternals.getDelegatorFullInfo(delegationID);
            if (!smOk) continue;
            if (status == 3) {
            ...
            } else if (status == 2 && startTime != 0 && startTime
    <= maturityCutoff && smAmount > 0) {
                ...

            }
>>>         // else: non-Active status (PendingAdded, PendingRemoved already
    tracked, Unknown) — skip
        }
    }
```

However, a `PendingAdded` delegation can be immediately removed without ever becoming Active when its parent validator is in `PendingRemoved` or `Completed` status and the delegation's churn lock period has elapsed.

```
    function completeDelegatorRegistration(
        bytes32 delegationID,
        uint32 messageIndex,
        uint32 uptimeMessageIndex
    ) external nonReentrant {
        StakingManagerStorage storage $ = _getStakingManagerStorage();

        Delegator memory delegator = $._delegatorStakes[delegationID];
        bytes32 validationID = delegator.validationID;
        Validator memory validator = $._manager.getValidator(validationID);

        // Ensure the delegator is pending added. Since anybody can call
    this function once
        // delegator registration has been initiated, we need to make sure
    that this function is only
        // callable after that has been done.
        if (delegator.status ≠ DelegatorStatus.PendingAdded) {
            revert InvalidDelegatorStatus(delegator.status);
        }

        // Once the validator has started exiting, this delegation can no
    longer become active.
        // Fall through directly to the removal/refund path instead of
    activating it.
        if (
>>>         validator.status == ValidatorStatus.Completed ||
            validator.status == ValidatorStatus.PendingRemoved
        ) {
            return _completeDelegatorRemoval(delegationID);
        }
```

Because `_processOperatorDelegationsForRemoval` skips `PendingAdded` entirely, these removable delegations are never accounted for. The vault will under-count available liquidity during withdrawal processing, potentially blocking withdrawals that should be fulfillable.

Recommendations:

Within `_processOperatorDelegationsForRemoval`, add a handling branch for status == PendingAdded.

Kite AI: Acknowledged. We reproduced the scenario in tests: withdrawals are delayed until

someone calls completeDelegatorRegistration manually (permissionless). No funds permanently at risk, but recovery depends on relayer liveness. Foundry tests in `@06dfcae ...`

[L-3] _processOperatorValidatorsForRemoval consumes scan budget for already pending validators

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L1100

Description:

`_processOperatorValidatorsForRemoval` increments scans before checking whether a validator is already pending removal:

```
    function _processOperatorValidatorsForRemoval(
        StakingVaultStorageLib.StakingVaultStorage storage $,
        address operatorAddr,
        uint256 remainingTarget,
        uint64 maturityCutoff,
        uint256 removalsThisCall,
        uint256 maxScans
    ) internal returns (uint256 contribution, uint256 removals,
    uint256 scans) {
        ...
            unchecked {
>>>             ++scans;
            }

            bytes32 validationID = $.operatorValidators[operatorAddr].at(i);

            if ($.validatorPendingRemoval[validationID]) continue;
        ...
```

This is inconsistent with `_processOperatorDelegationsForRemoval`, which explicitly defers the scan increment until after the analogous skip check:

```
        function _processOperatorDelegationsForRemoval(
        StakingVaultStorageLib.StakingVaultStorage storage $,
        address operatorAddr,
```

```
        uint256 remainingTarget,
        uint64 maturityCutoff,
        uint256 removalsThisCall,
        uint256 maxScans
    ) internal returns (uint256 contribution, uint256 removals,
    uint256 scans) {
        uint256 delLen = $.operatorDelegations[operatorAddr].length();
        for (uint256 j = delLen; j > 0 && contribution < remainingTarget;) {
            unchecked {
                --j;
            }
            if (removalsThisCall + removals
    >= StakingVaultStorageLib.MAX_REMOVALS_PER_CALL || scans >= maxScans) {
                break;
            }

            bytes32 delegationID
    = $.operatorDelegations[operatorAddr].at(j);

            // Skip vault-initiated pending removals without consuming scan
    budget
            if ($.delegatorRemovalInitiatedEpoch[delegationID] ≠ 0)
    continue;

            unchecked {
>>>             ++scans;
            }
            ...
```

Recommendations:

Move the ++scans increment to after the `validatorPendingRemoval` check, mirroring the pattern used in `_processOperatorDelegationsForRemoval`.

Kite AI: Resolved with `@271e30a ...`

Zenith: Verified.

[L-4] Validator removal initiated before all delegations are removed

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L890

Description:

`_selectAndRemoveStake` is structured in two sequential phases: Phase 1 scans all operators' delegations to meet the removal target, and Phase 2 falls back to removing validators for any operator whose delegations fell short.

```
        function _selectAndRemoveStake(
        uint256 amountNeeded
    ) internal returns (uint256 removalsInitiated) {
        ...

        uint256[] memory contributions = new uint256[](ctx.activeCount);
        uint256 removalsThisCall;
        uint256 delegationScansThisCall;

        // Phase 1: Process delegations for all operators
        for (
            uint256 i;
            i < ctx.activeCount && removalsThisCall <
    StakingVaultStorageLib.MAX_REMOVALS_PER_CALL
>>>             && delegationScansThisCall <
    StakingVaultStorageLib.MAX_DELEGATION_SCAN_PER_CALL;

        ) {
            ...
        }

        // Phase 2: Fallback to validators when delegations didn't meet
    target
        ...
    }
```

The Phase 1 outer loop exits as soon as `delegationScansThisCall` reaches `MAX_DELEGATION_SCAN_PER_CALL` (300). Because this cap is checked on the outer loop across all operators, it can terminate Phase 1 while some operators have not yet had any of their delegations scanned. When Phase 2 then begins, `contributions[i]` for those skipped operators will be zero (or below their `targetShares[i]`), causing validators to be marked for removal even though active delegations for that operator were never processed. This violates the intended design that all of an operator's delegations must be removed before any of its validators are removed. A validator removed can stop still-live delegations from earning beyond the validator end time.

This scenario is most likely to occur when many operators each have large numbers of active delegations, exhausting the 300-scan budget before all operators' delegations are processed.

Recommendations:

Remove all delegators before remove any validator.

Kite AI: Resolved with `@19d9a212b0 ...`

Zenith: Verified.

[L-5] Wrong maturity cutoff for validators

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- StakingVaultOperations.sol#L823-L824

Description:

When selecting a validator for stake removal, `_selectAndRemoveStake` computes `maturityCutoff` using `StakingVaultInternals.getMinimumStakeDuration`, which reads the current global `_minimumStakeDuration` from the `StakingManager`.

```
    function _selectAndRemoveStake(
        uint256 amountNeeded
    ) internal returns (uint256 removalsInitiated) {
        ...

>>>     uint64 minDuration = StakingVaultInternals.getMinimumStakeDuration();
>>>     ctx.maturityCutoff = uint64(block.timestamp) > minDuration ?
    uint64(block.timestamp) - minDuration : 0;

        ...
        // Phase 2: Fallback to validators when delegations didn't meet
    target
        uint256 validatorScansThisCall;
        for (
            uint256 i;
            i < ctx.activeCount && removalsThisCall <
    StakingVaultStorageLib.MAX_REMOVALS_PER_CALL
                && validatorScansThisCall <
    StakingVaultStorageLib.MAX_VALIDATOR_SCAN_PER_CALL;

        ) {
            ...
                (uint256 valContrib, uint256 valRemovals, uint256 valScans)
    = _processOperatorValidatorsForRemoval(
>>>                 $, operatorAddr, remainingTarget, ctx.maturityCutoff,
    removalsThisCall, perOpValidatorScanBudget
```

```
                );
        ...
    }
```

However, each validator stores its own per-validator `minStakeDuration` at registration time, which is what `_initiatePoSValidatorRemoval` actually enforces:

```
    function _initiatePoSValidatorRemoval(
        bytes32 validationID,
        bool includeUptimeProof,
        uint32 messageIndex
    ) internal {
        ...
        // Check that minimum stake duration has passed.
        if (
            validator.endTime <
            validator.startTime +
>>>             $._posValidatorInfo[validationID].minStakeDuration
        ) {
            revert MinStakeDurationNotPassed(validator.endTime);
        }

        ...
    }
```

And the global `_minimumStakeDuration` can be updated by StakingManager owner:

```
    function _updateStakingConfig(
        uint256 minimumStakeAmount,
        uint256 maximumStakeAmount,
        uint64 minimumStakeDuration,
        uint16 minimumDelegationFeeBips,
        uint8 maximumStakeMultiplier
    ) internal {
        ...

        // Update storage
        $._minimumStakeAmount = minimumStakeAmount;
        $._maximumStakeAmount = maximumStakeAmount;
>>>     $._minimumStakeDuration = minimumStakeDuration;
        $._minimumDelegationFeeBips = minimumDelegationFeeBips;
        $._maximumStakeMultiplier = maximumStakeMultiplier;
    }
```

This creates a mismatch in two scenarios:

- If global `_minimumStakeDuration` is decreased, the function would treat validators as mature when they are not, wasting scan budget.
- If global `_minimumStakeDuration` is increased, validators that have genuinely satisfied their individual stake commitment are incorrectly excluded from the eligible set, skewing the selection and potentially preventing removals from completing.

Recommendations:

Store maturity of each validator when it's intialized instead of deriving `maturityCutoff` from the mutable global `_minimumStakeDuration`.

Kite AI: Acknowledged. Both mismatch directions are bounded: wasted scan budget if lowered, slower auto-removal if raised. We can coordinate with the SM admin to clear the withdrawal queue before any duration change, and use forceRemoveValidator to handle specific validators if needed.

[L-6] initiateValidatorRemoval hardcodes includeUptimeProof to false

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L162

Description:

`initiateValidatorRemoval` unconditionally passes `includeUptimeProof = false` and `messageIndex = 0` when calling `forceInitiateValidatorRemoval` on the `StakingManager`:

```
    function initiateValidatorRemoval(
        bytes32 validationID
    ) external nonReentrant onlyOperator {
        ...

>>>     $.stakingManager.forceInitiateValidatorRemoval(validationID, false,
    0);

        ...
    }
```

When `includeUptimeProof` is false, the StakingManager falls back to the latest cached uptime value to calculate the validator's staking reward at exit. This value may be stale relative to the validator's actual accumulated uptime at the time of removal.

Operators have no way to submit a fresh uptime proof at the point of removal. This means they will receive a lower reward than it is entitled to.

Recommendations:

Add `includeUptimeProof` and `messageIndex` as parameters to `initiateValidatorRemoval`, and forward them directly to `forceInitiateValidatorRemoval`.

Kite AI: Acknowledged. For operator-initiated removal, operators can call submitUptimeProof on the SM independently before triggering removal. For admin

(forceRemoveValidator) and permissionless (prepareWithdrawals) paths, the operator has no advance notice, so the last submitted proof is used. Reward loss is bounded by staleness duration and operators are expected to submit proofs frequently. Due to size limitation we will keep this out.

[L-7] Inconsistent debt reference points in _selectAndRemoveStake can increase exit debt despite over-contribution

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- StakingVaultOperations.sol#L944-L973

Description:

The debt accounting loop in `_selectAndRemoveStake` uses two different reference points asymmetrically:

- Debt reduction is gated against `baseShare`
- Debt accrual is gated against `ctx.targetShares[i]`

This creates an inconsistency when `baseShare < contributions[i] < ctx.targetShares[i]`. In this range, the operator contributed more than their base allocation (which should reduce or hold steady their debt), yet the shortfall against `targetShares[i]` still triggers debt accrual. The net result is that `totalExitDebt` can increase even when an operator has exceeded their fair-share contribution.

```
        for (uint256 i; i < ctx.activeCount;) {
            address operatorAddr = ctx.activeOperators[i];
            uint256 baseShare = ctx.totalAllocationBips > 0
                ? (ctx.effectiveNeeded
    * $.operators[operatorAddr].allocationBips) / ctx.totalAllocationBips
                : 0;

>>>         if (contributions[i] > baseShare) {
                uint256 paydown = contributions[i] - baseShare;
                uint256 currentDebt = $.operatorExitDebt[operatorAddr];
                uint256 reduction = paydown < currentDebt ? paydown :
    currentDebt;
                if (reduction > 0) {
                    $.operatorExitDebt[operatorAddr] -= reduction;
                    $.totalExitDebt -= reduction;
                    emit
```

```
    IStakingVaultOperations.StakingVault__ExitDebtReduced(
                        operatorAddr, reduction,
    $.operatorExitDebt[operatorAddr]
                    );
                }
            }
>>>         if (contributions[i] < ctx.targetShares[i]) {
                uint256 shortfall = ctx.targetShares[i] - contributions[i];
                $.operatorExitDebt[operatorAddr] += shortfall;
                $.totalExitDebt += shortfall;
                emit IStakingVaultOperations.StakingVault__ExitDebtRecorded(
                    operatorAddr, shortfall, $.operatorExitDebt[operatorAddr]
                );
            }
            unchecked {
                ++i;
            }
        }
```

Since both if branches are independent, they can both execute in the same iteration, partially paying down debt while simultaneously accruing more, leaving debt flat or higher than before.

Recommendations:

Use `baseShare` exclusively to determine whether debt is reduced or accrued. `ctx.targetShares[i]` should only be used as the removal target during the scanning phases (Phases 1 and 2), not for debt settlement.

Kite AI: Acknowledged. Design trade-off and will documenting for now given the window is so small.

[L-8] Operators skipped by caps unfairly accrue exit debt

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L919
- StakingVaultOperations.sol#L889

Description:

In `_selectAndRemoveStake`, Phases 1 and 2 iterate over active operators but halt early once `MAX_REMOVALS_PER_CALL` or their respective scan caps are reached. Any operator not reached by the loop has its `contributions[i]` left at zero, because pendingCredit (`operatorPriorEpochPendingAmount + operatorCurrentEpochPendingAmount`) is only applied inside the loop body:

```
    function _selectAndRemoveStake(
        uint256 amountNeeded
    ) internal returns (uint256 removalsInitiated) {
        ...

        // Phase 1: Process delegations for all operators
        for (
            uint256 i;
>>>         i < ctx.activeCount && removalsThisCall <
    StakingVaultStorageLib.MAX_REMOVALS_PER_CALL
                && delegationScansThisCall <
    StakingVaultStorageLib.MAX_DELEGATION_SCAN_PER_CALL;

        ) {
            address operatorAddr = ctx.activeOperators[i];
>>>         uint256 pendingCredit =
                $.operatorPriorEpochPendingAmount[operatorAddr]
    + $.operatorCurrentEpochPendingAmount[operatorAddr];
            ...
        }

        // Phase 2: Fallback to validators when delegations didn't meet
    target
```

```
        uint256 validatorScansThisCall;
        for (
            uint256 i;
>>>         i < ctx.activeCount && removalsThisCall <
    StakingVaultStorageLib.MAX_REMOVALS_PER_CALL
                && validatorScansThisCall <
    StakingVaultStorageLib.MAX_VALIDATOR_SCAN_PER_CALL;

        ) {
            ...
    }
```

The debt-accrual loop in Phase 3, however, is unconditional, it iterates all `activeCount` operators regardless of whether they were evaluated:

```
for (uint256 i; i < ctx.activeCount;) {
    ...
    if (contributions[i] < ctx.targetShares[i]) {
        uint256 shortfall = ctx.targetShares[i] - contributions[i];
        $.operatorExitDebt[operatorAddr] += shortfall; // accrued even for
    skipped operators
        $.totalExitDebt += shortfall;
        ...
    }
}
```

A skipped operator will always show `contributions[i] == 0`, creating an apparent shortfall equal to its full `targetShares[i]`, even if its `pendingCredit` would have partially or fully satisfied the target. This causes unfair exit debt accrual that can trigger the 50% debt freeze threshold and block new stake deployments.

Recommendations:

Accrue debt for operators that were actually evaluated to exhaustion, and not for operators whose shortfall exists only because the call hit a global scan/remove cap.

Kite AI: Resolved with `@19d9a21 ...` and `@3540c38 ...`

Zenith: Verified.

[L-9] requestWithdrawal fee deducts vaultAccountedBalance

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingVault.sol#L380

Description:

In `requestWithdrawal`, when a `withdrawalRequestFee` > 0 is configured, the fee is immediately deducted from `vaultAccountedBalance` and transferred out of the contract before the rest of the request is queued:

```
    function requestWithdrawal(
        uint256 shares
    ) external nonReentrant whenNotPaused returns (uint256 requestId) {
        ...

        uint256 fee = $.withdrawalRequestFee;
        if (fee > 0) {
            if (stakeAmount <= fee) revert StakingVault__InvalidAmount();
            stakeAmount -= fee;
>>>         $.vaultAccountedBalance -= fee;
            (bool feeSuccess,) = $.protocolFeeRecipient.call{value:
    fee}("");
            if (!feeSuccess) {
                $.vaultAccountedBalance += fee;
                $.pendingProtocolFees += fee;
                emit
    IStakingVaultOperations.StakingVault__ProtocolFeeEscrowed(fee,
    $.pendingProtocolFees);
            }
        }

        ...
    }
```

Consider a scenario when:

- the vault has no free liquidity i.e., `getAvailableStake == 0` but `vaultAccountedBalance > 0` because it is entirely backing `claimableWithdrawalStake`, the fee deduction in `requestWithdrawal` breaks this invariant. The fee is successfully sent to `protocolFeeRecipient`, removing ETH from the contract that was reserved for existing claimants.
- As a consequence, when one of those older claimants calls `claimWithdrawal`, the deduction step in `_sendWithdrawalOrEscrow`: `vaultAccountedBalance -= stakeAmount;` will revert due to an arithmetic underflow, even though `processEpoch` had already marked the request as claimable and the user has been waiting for their funds.

A secondary edge case occurs when `vaultAccountedBalance` is positive but smaller than the `withdrawalRequestFee` itself. In this state `requestWithdrawal` reverts on the fee subtraction, blocking new withdrawal requests from being queued despite the vault not being paused.

Recommendations:

It's recommended to either:

- only send the fee when `getAvailableStake() ≥ fee` so that no ETH backing claimable withdrawals is removed.
- collect the fee inside `claimWithdrawal` instead of at request time. At that point, the ETH has already been returned to the vault's liquid balance, so deducting the fee does not touch reserved claimable funds.

Kite AI: Resolved with `@ee03fa2af0 ...`

Zenith: Verified.

[L-10] MAX_OPERATOR_FEE_BIPS cap can block validator registration

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVault.sol#L608

Description:

`setOperatorFeeBips` enforces a hard cap of `MAX_OPERATOR_FEE_BIPS` (2000 bips / 20%) on the operator fee:

```
    function setOperatorFeeBips(
        uint256 bips
    ) external onlyRole(StakingVaultStorageLib.VAULT_ADMIN_ROLE) {
>>>     if (bips > StakingVaultStorageLib.MAX_OPERATOR_FEE_BIPS) {
            revert StakingVault__InvalidFee(bips);
        }
        uint16 minFeeBips = _getMinimumDelegationFeeBips();
        if (bips < uint256(minFeeBips)) {
            revert StakingVault__InvalidFee(bips);
        }

        ...
    }
```

However, the `StakingManager` contract allows `minimumDelegationFeeBips` to be set anywhere up to `MAXIMUM_DELEGATION_FEE_BIPS` (10000 bips / 100%):

```
    uint16 public constant MAXIMUM_DELEGATION_FEE_BIPS = 10000;

    function _updateStakingConfig(
        uint256 minimumStakeAmount,
        uint256 maximumStakeAmount,
        uint64 minimumStakeDuration,
        uint16 minimumDelegationFeeBips,
        uint8 maximumStakeMultiplier
```

```
    ) internal {
        StakingManagerStorage storage $ = _getStakingManagerStorage();

        // Validate parameters
        if (
            minimumDelegationFeeBips == 0 ||
>>>         minimumDelegationFeeBips > MAXIMUM_DELEGATION_FEE_BIPS
        ) {
            revert InvalidDelegationFee(minimumDelegationFeeBips);
        }
        ...
    }
```

If the `StakingManager` owner raises `minimumDelegationFeeBips` above 20%, `setOperatorFeeBips` becomes uncallable, any value below the new minimum reverts due to the floor check, and any value above 20% reverts due to the cap.

Since `initiateValidatorRegistration` passes `operatorFeeBips` directly as the `delegationFeeBips` to the `StakingManager`, and the `StakingManager` will reject fees below its updated minimum, all new validator registrations are blocked until the fee configuration is resolved.

```
        function initiateValidatorRegistration(
        bytes memory nodeID,
        bytes memory blsPublicKey,
        PChainOwner memory remainingBalanceOwner,
        PChainOwner memory disableOwner,
        uint256 amount
    ) external nonReentrant onlyOperator returns (bytes32 validationID) {
        ...

        uint64 minStakeDuration
    = StakingVaultInternals.getMinimumStakeDuration();
>>>     uint16 delegationFeeBips = uint16($.operatorFeeBips);

        $.vaultAccountedBalance -= amount;
        validationID = $.stakingManager.initiateValidatorRegistration{value:
    amount}(
            nodeID,
            blsPublicKey,
            remainingBalanceOwner,
            disableOwner,
            delegationFeeBips,
>>>         minStakeDuration,
            address(this)
        );
```

```
        ...
    }
```

Recommendations:

Raise `MAX_OPERATOR_FEE_BIPS` to 10000 (100%) to align with `MAXIMUM_DELEGATION_FEE_BIPS` in the `StakingManager`.

Kite AI: Acknowledged.

MAX_OPERATOR_FEE_BIPS = 2000 (20%) is a deliberate product cap. The initializer validates the constraint at deploy time (StakingVault.sol:1175). The SM admin is trusted.

Impact is limited to blocking new validator registrations; existing validators, delegations, withdrawals, and harvesting are unaffected. Documented as an operational constraint. SM admin must not raise minimumDelegationFeeBips above the vault's MAX_OPERATOR_FEE_BIPS after vault deployment. As a last resort, the vault is UUPS upgradeable, a new implementation with a higher cap can be deployed to unblock registrations.

[L-11] `requestWithdrawal` lacks slippage protection

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVault.sol#L372-L425

Description:

`deposit()` has a `minShares` parameter but `requestWithdrawal()` has no equivalent `minStakeAmount` parameter, the user burns shares and accepts whatever `_sharesToStake(shares)` returns at execution time.

```
function requestWithdrawal(
    uint256 shares
) external nonReentrant whenNotPaused returns (uint256 requestId) {
    if (shares == 0) revert StakingVault__InvalidAmount();
    uint256 balance = balanceOf(msg.sender);
    if (balance < shares) {
        revert StakingVault__InsufficientBalance(shares, balance);
    }

    StakingVaultStorageLib.StakingVaultStorage storage $
    = StakingVaultStorageLib._getStorage();

    uint256 stakeAmount = _sharesToStake(shares);

    // ...
}
```

Since the `stakeAmount` is permanently locked at request time and determines the final ETH the user receives at claim, any rate decrease or fee changes between submission and execution is an irreversible loss for the withdrawer.

Recommendations:

Add a `minStakeAmount` parameter to `requestWithdrawal`.

Kite AI: Acknowledged. The rate conversion in `_sharesToStake` is computed and committed in the same transaction, so the risk is limited to mempool ordering between submission and inclusion. We agree a `minStakeAmount` parameter would be a nice safeguard and plan to add it in a future version.

[L-12] Withdrawal amount may exclude unharvested rewards

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVault.sol#L383

Description:

`getTotalPooledStake()` excludes unharvested rewards sitting in the `StakingManager`. `requestWithdrawal()` locks the `stakeAmount` at call time using the current `_sharesToStake()` calculation.

This will burn shares at understated price, lock in a withdrawal amount that doesn't include pending rewards.

Recommendations:

Consider to trigger `harvest` before calculating `stakeAmount` inside `requestWithdrawal`.

Kite AI: Acknowledged. We consider keeper and reward frequency management sufficient to keep the claimable backlog small enough that extractable value is negligible.

4.4 Informational

A total of 5 informational findings were identified.

[I-1] DEFAULT_ADMIN_ROLE can call initiateDelegatorRemoval

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L350

Description:

The vault enforces a deliberate separation between operator-initiated and admin-initiated delegator removals. Operators initiate removals through `initiateDelegatorRemoval`, while emergency admin removals are handled exclusively by `forceRemoveDelegator`, which is gated behind `VAULT_ADMIN_ROLE`.

However, the access control check in `initiateDelegatorRemoval` additionally permits the `DEFAULT_ADMIN_ROLE` (role ID bytes32(0)) to invoke the function:

```
    function initiateDelegatorRemoval(
        bytes32 delegationID
    ) external nonReentrant {
        ...

>>>     if (msg.sender ≠ info.operator &&
    !IAccessControl(address(this)).hasRole(bytes32(0), msg.sender)) {
            revert
    IStakingVault.StakingVault__NotDelegatorOperator(delegationID,
    msg.sender);
        }

        ...
    }
```

It violates the principle of least privilege and blurs the role boundaries of the protocol.

Recommendations:

Remove the `DEFAULT_ADMIN_ROLE` bypass from `initiateDelegatorRemoval`.

Kite AI: Resolved with `@7ac45f1 ...`

Zenith: Verified.

[I-2] Externally removed delegations may be missed in contribution accounting

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L983-L1075

Description:

`_processOperatorDelegationsForRemoval` iterates over delegations in reverse order and exits early once `contribution ≥ remainingTarget` or scan/removal limits are reached. During this scan, if a delegation is found in status 3 (externally initiated `PendingRemoved`), the vault adopts it and credits its principal toward contribution.

However, because the loop terminates early, any externally removed delegation that appears at a lower index beyond the early exit point is never adopted and its principal is never credited:

```
        function _processOperatorDelegationsForRemoval(
        StakingVaultStorageLib.StakingVaultStorage storage $,
        address operatorAddr,
        uint256 remainingTarget,
        uint64 maturityCutoff,
        uint256 removalsThisCall,
        uint256 maxScans
    ) internal returns (uint256 contribution, uint256 removals,
    uint256 scans) {
        uint256 delLen = $.operatorDelegations[operatorAddr].length();
        for (uint256 j = delLen; j > 0 && contribution < remainingTarget;) {
            ...
            if (status == 3) {
                // Adopt externally-initiated PendingRemoved we haven't
    tracked
                _recordRemovalInFlight($, operatorAddr, delegationID,
    principal, true);
>>>             contribution += principal;
                unchecked {
                    ++removals;
```

```
                }
                emit
IStakingVaultOperations.StakingVault__DelegatorRemovalAdopted(
                    operatorAddr, delegationID, principal
                );
            } else if (status == 2 && startTime != 0 && startTime
<= maturityCutoff && smAmount > 0) {
                ...
        }
}
```

Because these delegations are already exiting at the `StakingManager` level, their stake will leave regardless. Failing to credit them means the vault may initiate additional removals unnecessarily to meet the same target, resulting in over-removal beyond what is actually required.

This may inflate `operatorExitDebt` and `totalExitDebt` relative to what a full scan would have produced, creating an unfair debt accounting outcome for the operator.

Recommendations:

Before the main removal loop, perform a dedicated prepass over all delegations to identify and adopt any externally-initiated PendingRemoved entries (status 3).

Kite AI: Acknowledged. The over-removal self-corrects within one epoch. Missed delegations are already exiting on the P-Chain and their principal returns on completion. The prepass requires an additional unbounded loop with external SM calls per delegation to check status, plus the adoption logic. We accept the temporary over-removal over the added code and unbounded iteration.

[I-3] Malicious protocolFeeRecipient can break vault accounting

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L1188

Description:

`harvest` opens the `isReceivingManagerFunds` gate for the duration of its entire execution, allowing the vault's `receive` to accept ETH from the `StakingManager`:

```
    function harvest() external nonReentrant returns (uint256 totalRewards)
    {
        StakingVaultStorageLib.StakingVaultStorage storage $
    = StakingVaultStorageLib._getStorage();
>>>     $.isReceivingManagerFunds = true;
        uint256 opLen = $.operatorSet.length();
        for (uint256 i; i < opLen;) {
            totalRewards += _harvestOperatorValidators($, i, 0,
    type(uint256).max);
            totalRewards += _harvestOperatorDelegators($, i, 0,
    type(uint256).max);
            unchecked {
                ++i;
            }
        }
>>>     $.isReceivingManagerFunds = false;
    }
```

Inside the loop, `_harvestOperatorValidators` and `_harvestOperatorDelegators` each call `_sendProtocolFee`, which performs an external .call to the `protocolFeeRecipient`:

```
     function _harvestOperatorValidators(
        StakingVaultStorageLib.StakingVaultStorage storage $,
        uint256 operatorIndex,
        uint256 start,
```

```
        uint256 batchSize
    ) internal returns (uint256 totalRewards) {
        ...

        if (totalProtocolFee > 0) {
>>>         _sendProtocolFee($, totalProtocolFee);
        }

        ...
    }
```

If `protocolFeeRecipient` is a malicious contract, while the gate is still open, it can trigger removal completions (`completeValidatorRemoval`, `completeDelegatorRemoval`) or reward claims (`claimValidatorRewardsFor`, `claimDelegatorRewardsFor`). These cause the `StakingManager` to push native token into the vault. Because `isReceivingManagerFunds` is still `true`, the vault's `receive` accepts the ETH without going through the normal accounting paths that update `vaultAccountedBalance`. The native token silently inflates `address(this).balance` while remaining invisible to the vault's share-price and liquidity calculations, breaking the vault accounting.

Recommendations:

Accumulate total protocol fees across the entire harvest loop and defer the `_sendProtocolFee` call until after `isReceivingManagerFunds` is set back to `false`.

Kite AI: Acknowledged. The attack is valid, but deferring `_sendProtocolFee` requires quite some change, and `protocolFeeRecipient` is set by VAULT_ADMIN_ROLE, a trusted role. We consider a malicious admin has far simpler ways to rug the vault.

[I-4] completeValidatorRegistration does not verify validator belongs to the vault

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingVaultOperations.sol#L138-L144

Description:

`completeValidatorRegistration` forwards `messageIndex` directly to the staking manager and emits a `StakingVault__ValidatorRegistrationCompleted` event for the returned `validationID`, without checking whether that validator was registered through this vault.

```
    function completeValidatorRegistration(
        uint32 messageIndex
    ) external nonReentrant returns (bytes32 validationID) {
        StakingVaultStorageLib.StakingVaultStorage storage $
    = StakingVaultStorageLib._getStorage();
>>>     validationID
    = $.stakingManager.completeValidatorRegistration(messageIndex);
        emit IStakingVaultOperations.
            StakingVault__ValidatorRegistrationCompleted(validationID);
    }
```

Because the underlying `completeValidatorRegistration` on the staking manager is permissionless, a caller can supply a `messageIndex` for a validator belonging to a different vault, causing this vault to emit a misleading completion event for a foreign validator.

Recommendations:

Apply the same ownership check used in `completeValidatorRemoval`:

```
validationID = $.stakingManager.completeValidatorRegistration(messageIndex);
if ($.validatorToOperator[validationID] == address(0)) {
    revert IStakingVault.StakingVault__ValidatorNotFound(validationID);
}
```

Kite AI: Resolved with `@c553101cb6 ...`

Zenith: Verified.

[I-5] Insolvent vault has no ETH injection mechanism causing permanent deposit freeze with no admin recovery path

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingVault.sol#L356

Description:

If `getTotalPooledStake()` returns 0 while `totalSupply() > 0`, the vault enters an insolvent state where deposits are blocked.

```
    function deposit(
        uint256 minShares
    ) external payable nonReentrant whenNotPaused returns (uint256 shares) {
        if (msg.value == 0) revert StakingVault__InvalidAmount();

        uint256 preDepositStake = getTotalPooledStake();
        if (totalSupply() > 0 && preDepositStake == 0)
    revert StakingVault__Insolvent();
// ...
```

The `receive()` function rejects ALL non-SM ETH transfers. There is no admin-callable function to inject ETH and increase `vaultAccountedBalance` without minting shares.

Recommendations:

Consider to add admin-callable function to inject ETH and increase `vaultAccountedBalance` in case of insolvent vault.

Kite AI: Acknowledged. Since there's no-slashing chain, reaching insolvency is difficult: principal always returns in full from the P-Chain, and fees only consume reward income. We consider the deposit guard is enough defense. If it ever triggers, recovery is possible via UUPS upgrade.